BEE 4750 Lab 2: Monte Carlo

Name: Parker Catena

ID: pdc76 - 5465883

Due Date

Thursday, 10/02/25, 9:00pm

Setup

The following code should go at the top of most Julia scripts; it will load the local package environment and install any needed packages. You will see this often and shouldn't need to touch it.

```
import Pkg
Pkg.activate(".")
Pkg.instantiate()
```

```
  Activating project at `~/Lab2 Copy/lab-2-pdc76`
```

```
using Random # random number generation
using Distributions # probability distributions and interface
using Statistics # basic statistical functions, including mean
using Plots # plotting
```

Overview

In this lab, we will conduct a Monte Carlo experiment and explore how sensitive Monte Carlo estimates are to the underlying probability distribution(s).

You should always start any computing with random numbers by setting a "seed," which controls the sequence of numbers which are generated (since these are not *really* random, just "pseudorandom"). In Julia, we do this with the `Random.seed!()` function.

```
Random.seed!(1)
```

```
TaskLocalRNG()
```

It doesn't matter what seed you set, though different seeds might result in slightly different values. But setting a seed means every time your notebook is run, the answer will be the same.

> **Seeds and Reproducing Solutions**
>
> If you don't re-run your code in the same order or if you re-run the same cell repeatedly, you will not get the same solution. If you're working on a specific problem, you might want to re-use `Random.seed()` near any block of code you want to re-evaluate repeatedly.

Probability Distributions and Julia

Julia provides a common interface for probability distributions with the `Distributions.jl` package. The basic workflow for sampling from a distribution is:

1. Set up the distribution. The specific syntax depends on the distribution and what parameters are required, but the general call is the similar. For a normal distribution or a uniform distribution, the syntax is

   ```
   # you don't have to name this "normal_distribution"
   #   is the mean and   is the standard deviation
   normal_distribution = Normal( ,  )
   # a is the upper bound and b is the lower bound; these can be set to +Inf or -Inf for ar
   uniform_distribution = Uniform(a, b)
   ```

   There are lots of both univariate and multivariate distributions, as well as the ability to create your own, but we won't do anything too exotic here.

2. Draw samples. This uses the `rand()` command (which, when used without a distribution, just samples uniformly from the interval $[0, 1]$.) For example, to sample from our normal distribution above:

```
# draw n samples
rand(normal_distribution, n)
```

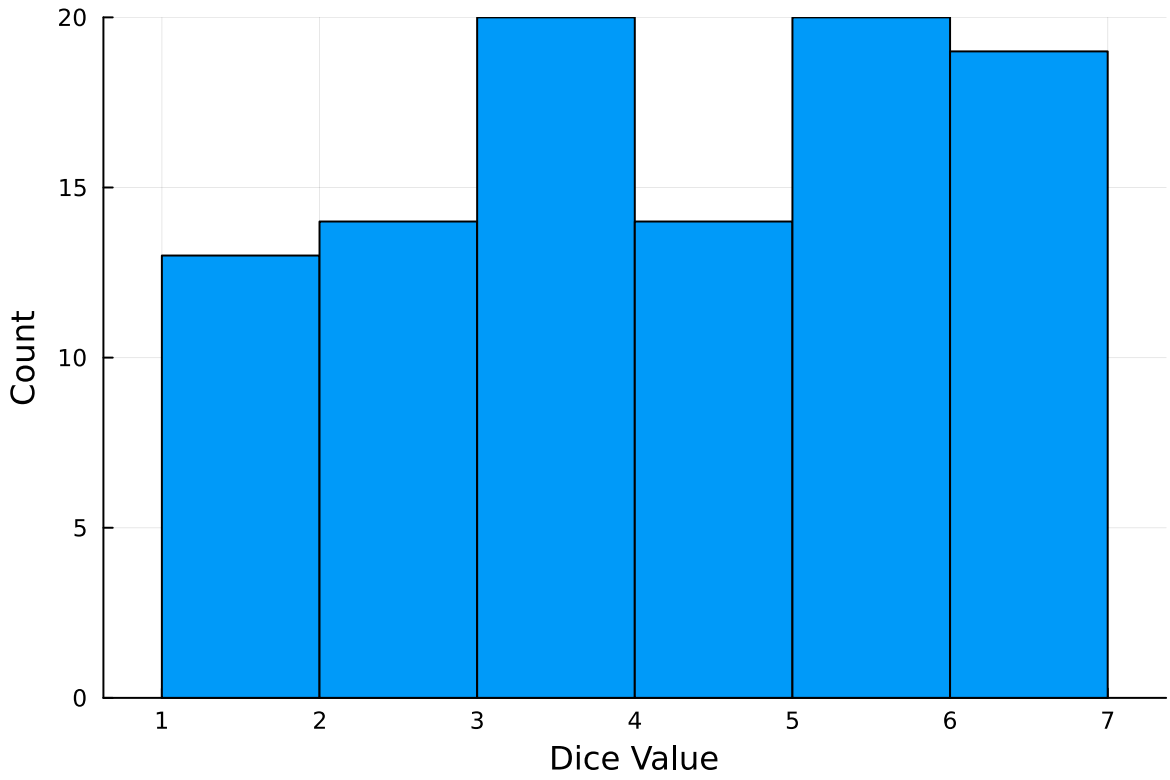
Putting this together, let's say that we wanted to simulate 100 six-sided dice rolls. We could use a Discrete Uniform distribution.

```
dice_dist = DiscreteUniform(1, 6) # can generate any integer between 1 and 6
dice_rolls = rand(dice_dist, 100) # simulate rolls
```

```
100-element Vector{Int64}:
 1
 3
 5
 4
 6
 2
 5
 5
 5
 2

 3
 6
 5
 5
 6
 3
 6
 6
 6
```

And then we can plot a histogram of these rolls:

```
histogram(dice_rolls, legend=:false, bins=6)
ylabel!("Count")
xlabel!("Dice Value")
```



Instructions

Remember to:

- Evaluate all of your code cells, in order (using a `Run All` command). This will make sure all output is visible and that the code cells were evaluated in the correct order.
- Tag each of the problems when you submit to Gradescope; a 10% penalty will be deducted if this is not done.

Problem

A common engineering problem is to quantify flood risk, which is typically computed by propagating a flood hazard distribution through a *depth-damage function* relating flood depths to economic damages. A reasonable depth-damage function for a house without a basement is a bounded logistic function,

$$d(h) = \mathbb{1}_{h>0} \frac{L}{1 + \exp(-k(h - h_0))},$$

where d is the damage as a percent of total structure value, h is the water depth (relative to the house's ground floor elevation) in m, $\mathbb{1}_{x>0}$ is the indicator function, L is the maximum loss in USD, k is the slope of the depth-damage relationship, and h_0 is the inflection point. We'll assume $L = \$200,000$, $k = 0.8$, and $h_0 = 3$.

For this problem, suppose that we have two different probability distributions characterizing annual maxima flood depths:

1. $h \sim LogNormal(1.2, 0.3)$;
2. $h \sim GeneralizedExtremeValue(3, 0.9, -0.15)$.

Plot histograms of samples from these distributions and conduct a Monte Carlo experiment for annual maximum flood damages using each. Answer the following questions:

- What are the Monte Carlo estimates of the expected value and the 99% quantile for the annual maximum damage the structure would suffer for each of these flood hazard distributions?
- How did you decide on your sample size?
- Why do you think the estimates differed or did not differ (you can also plot the depth-damage function to compare with the samples)?
- How might you decide (based on getting additional information, if needed) which distribution is "better"?

```
#Initialization
L = 200000 #max loss from damage in USD
k = 0.8 #Slope-depth parameter
ho = 3 #Inflection point

#Damage function
function damage(h)
    if h < 0
        d = 0
    else
        d = 1 * L/(1 + exp(-k * (h - ho)))
    return d
    end
end
```

```
damage (generic function with 1 method)
```

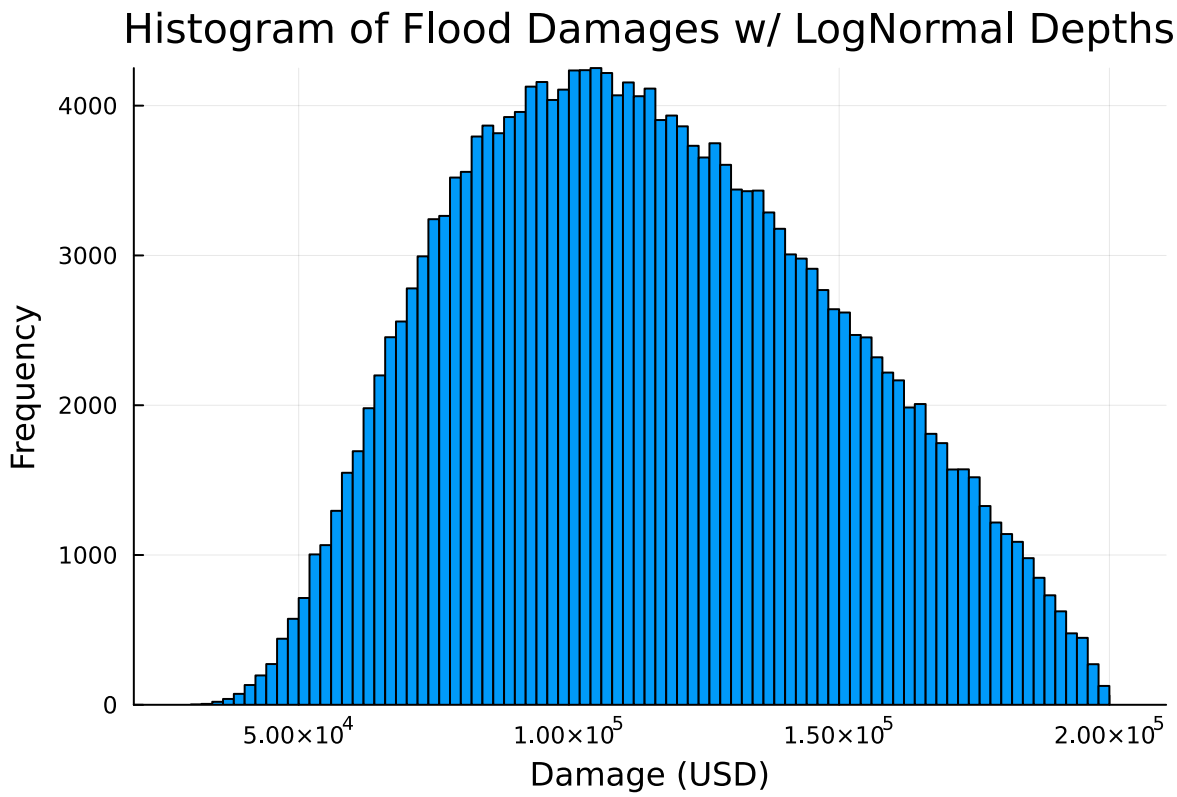
Above is the function for damage as laid out in the problem setup. It includes the initialized variables, L, K, h_o. It returns zero for $h < 0$, and the damage function evaluated at a height, h, otherwise.

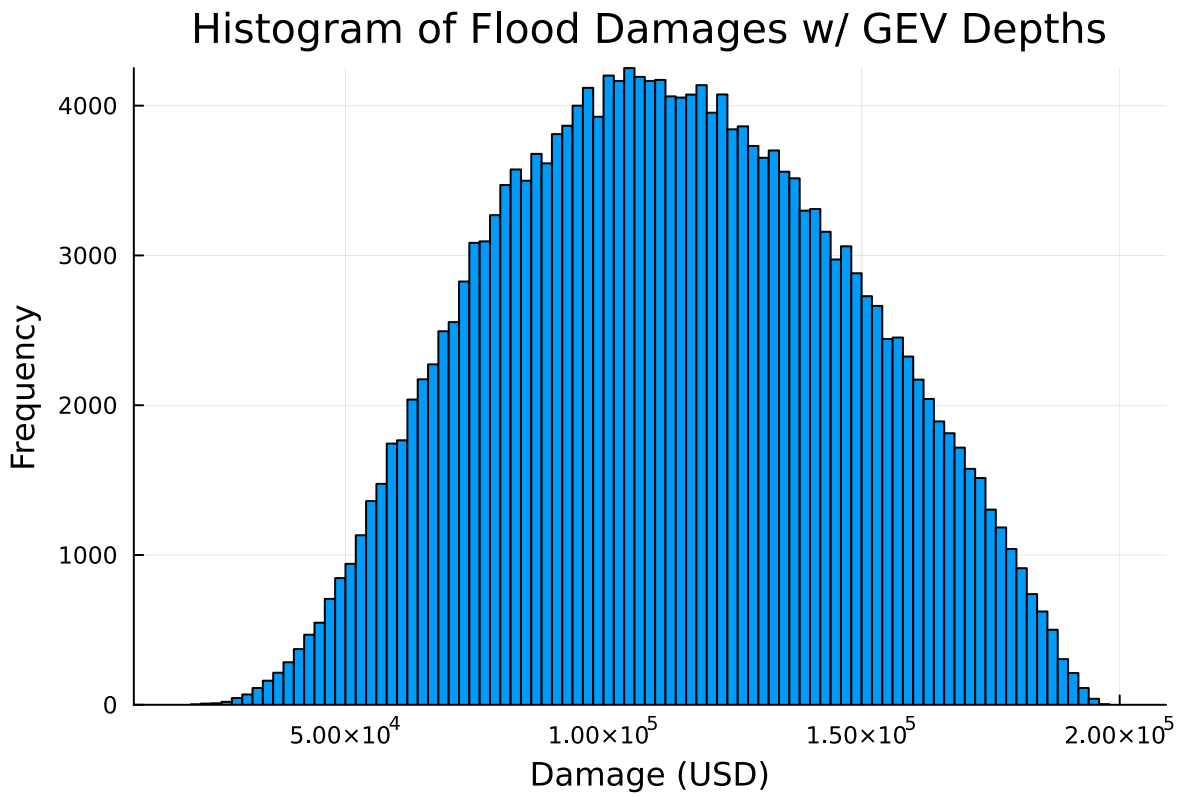
```
log_normal = LogNormal(1.2, 0.3) #log-normal distribution with log mean 1.2 and stddev 0.3
generalized_ev = GeneralizedExtremeValue(3, 0.9, -0.15) #generalized extreme value distributi
```

```
GeneralizedExtremeValue{Float64}( =3.0,  =0.9,  =-0.15)
```

Creates variables that call the probability distributions, LogNormal and GeneralizedExtremeValue.

```
#Simulate 200000 heights for each of the distributions
log_normal_heights = rand(log_normal, 200000)
log_damages = damage.(log_normal_heights)

generalized_ev_heights = rand(generalized_ev, 200000)
gev_damages = damage.(generalized_ev_heights)

# LogNormal damages histogram
log_graph = histogram(
    log_damages;
    xlabel="Damage (USD)",
    ylabel="Frequency",
    title="Histogram of Flood Damages w/ LogNormal Depths",
    legend=false)

# GEV damages histogram
ev_graph = histogram(
    gev_damages;
    xlabel="Damage (USD)",
    ylabel="Frequency",
    title="Histogram of Flood Damages w/ GEV Depths",
    legend=false)

display(log_graph)
display(ev_graph)
```


Histogram of Flood Damages w/ LogNormal Depths
Frequency
0
1000
2000
3000
4000
5.00×10^4
1.00×10^5
1.50×10^5
2.00×10^5
Damage (USD)



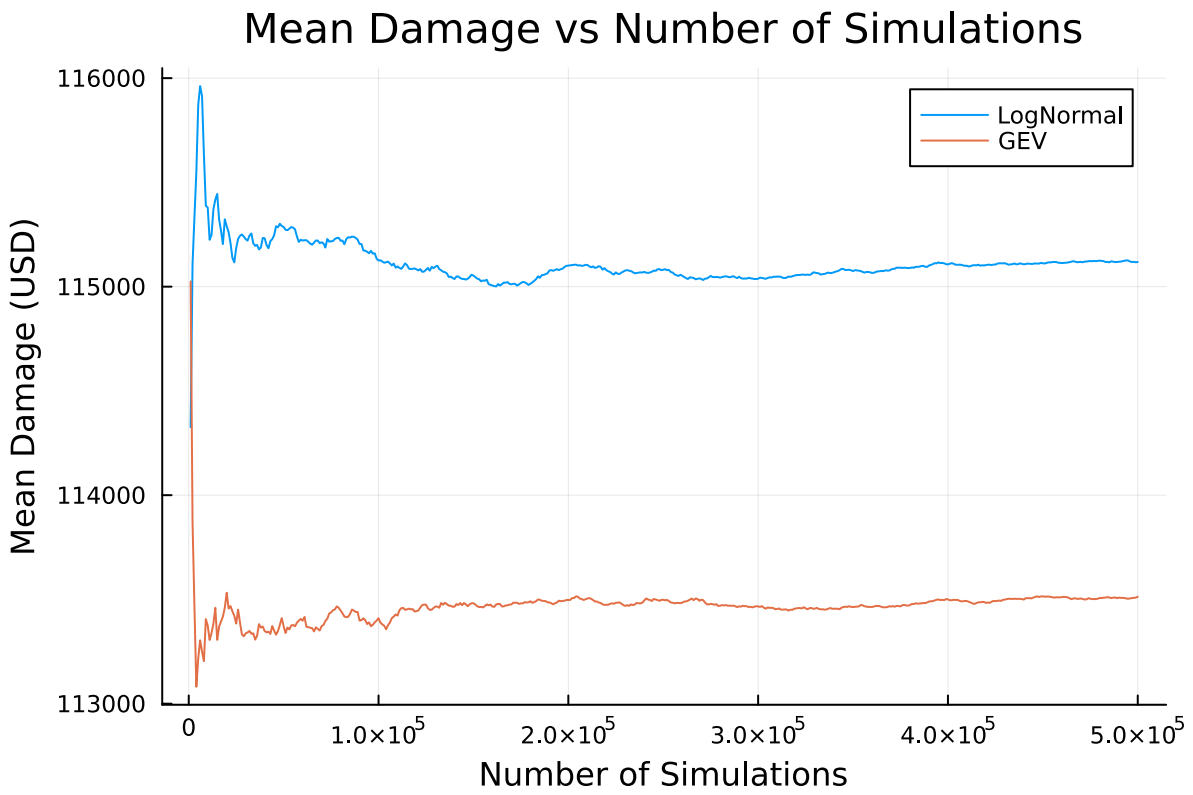
Generate the histograms for LogNormal and GeneralizedExtremeValues with frequency on the vertical axis and damage cost on the horizontal axis. Both of these histograms indicate a variance that is slightly greater than the mean, but overall have extremely similar distributions. The GEV distribution appears to be closer to normal in terms of the tail end of the data.

```
# Calculate the statistics for mean and 99th quantile
#Log statistics
log_mean = mean(log_damages)
log_quant = quantile(log_damages, 0.99)

#GEV statistics
gev_mean = mean(gev_damages)
gev_quant = quantile(gev_damages, 0.99)

#Print the statistics for both distributions
println("LogNormal Mean Damage: \$$(round(log_mean, digits=2))")
println("LogNormal 99th Percentile Damage: \$$(round(log_quant, digits=2))")
println("GeneralizedExtremeValue Mean Damage: \$$(round(gev_mean, digits=2))")
println("GeneralizedExtremeValue 99th Percentile Damage: \$$(round(gev_quant, digits=2))")
```

```
LogNormal Mean Damage: $115089.53
LogNormal 99th Percentile Damage: $189827.76
GeneralizedExtremeValue Mean Damage: $113446.94
GeneralizedExtremeValue 99th Percentile Damage: $183420.03
```

The mean for both LogNormal and GeneralizedExtremeValue are very similar at around $115,000.00. Similarly the 99th percentile for damage is around $185,000.00, but has slightly more variance in the values between LogNormal and GeneralizedExtremeValue.

```
#Take a sample of increasing size from 1000 to N to simulate convergence of the mean
N = 500000
sample_sizes = 1000:1000:N

#Simulate N heights for each of the distributions
log_normal_heights = rand(log_normal, N)
log_damages = damage.(log_normal_heights)
generalized_ev_heights = rand(generalized_ev, N)
gev_damages = damage.(generalized_ev_heights)

#Calculate and plot the mean damage as a function of sample size to see convergence
log_means = [mean(log_damages[1:n]) for n in sample_sizes]
gev_means = [mean(gev_damages[1:n]) for n in sample_sizes]
mean_graph = plot(
    sample_sizes,
    [log_means gev_means];
    xlabel="Number of Simulations",
    ylabel="Mean Damage (USD)",
    title="Mean Damage vs Number of Simulations",
    label=["LogNormal" "GEV"],
    legend=:topright)
display(mean_graph)
```



Above is a graph of the mean damage from LogNormal and GeneralizedExtremeValue distributions. What we can see is that they are unstable until around 200000 simulations at which point the mean of both distributions begins to converge on the mean values calculated by the "mean" function. With that being said, there is significant variance in the time to convergence between each trial. Some trials converge on mean values before 100000, whereas others do not converge until near 500000. When I was in class, I noted them converging closer to 100000, but for whatever reason when I opened the file later, it seems to take longer to converge. I am not sure what is causing this.

I believe the estimates differed between the two distributions as a result of their composition. The LogNormal function is the logarthmic distribution of a variable which skews to the right. The GeneralizedExtremeValue function is based on extreme values and is controlled by the variable for the shape of the distribution. I believe that the GeneralizedExtremeValue distribution is likely better for modeling flood risk if there is more likelihood of extreme events, however, the LogNormal function will likely fit more "average" floods. When reading scholarly articles on this subject, it seems that there is no "one-size fits all" solution. Therefore, to make a judgment on the best distribution, we would need empirical data to make strong claims about which is better.

References

Put any consulted sources here, including classmates you worked with/who helped you.

https://juliastats.org/Distributions.jl/v0.14/univariate.html - The documentation for Distribution.jl to help me understand the LogNormal and GeneralizedExtremeValue distributions.